

Fishbowl: An Event-Log Truthful Multi-Agent Office on a Raspberry Pi

Akash Premkumar

Abstract

Multi-agent demos often look busy while hiding a quiet lie: the on-screen characters move for atmosphere, while the real work happens in a chat log the viewer never sees. Fishbowl is a Stage 1 system for the opposite contract. A single Node orchestrator on a Raspberry Pi is the world clock. Every roughly 90–120 seconds it selects one employee, issues at most one model call, runs a narrow tool set, and appends structured events. The office canvas is a rendering of that event log plus a local layout file; sprites walk because an event occurred, not because an animation loop decided they should. A split-screen page puts the office on the left and, on the right, only the last green product build—never the working copy—behind a gate that syntax-checks and candidate-loads a write before promotion. The flagship product pane shows Meridian Office (docs first); the project name remains Fishbowl and the in-world studio is Meridian Desk. Secrets stay outside every path employees can read. A hard five-dollar UTC-day ceiling pauses ticks when spend would otherwise climb. Dry-run and replay modes keep the HDMI kiosk alive at zero token cost. This paper describes the system, the reasons a Pi is the right Stage 1 host rather than a laptop or cloud VM alone, the threat model for keys, and the observables we intend to measure (ticks, events, spend, green builds). We report no fabricated experimental tables. Where live sweep numbers are not yet collected, we say so and treat the manuscript as a design account plus a measurement plan. The work sits beside a sister measurement paper on purpose-built agent UI representations: Fishbowl chooses structured tools and an event log rather than screenshot grounding for the employees themselves.

1. Introduction

An always-on multi-agent “office” is easy to sketch and hard to trust. The usual failure mode is theatrical. Characters pace. Speech bubbles appear. A product panel shows something that may or may not be the build under discussion. The human watching cannot tell whether the motion is coupled to state. Once that coupling breaks, the display becomes a screensaver with better typography, and the eye notices within minutes.

Fishbowl is a private Stage 1 pilot that treats coupling as the product. The project name is Fishbowl. The fictional studio on the whiteboard is Meridian Desk. The right-hand product pane’s flagship is Meridian Office, a lightweight browser suite with a usable Docs surface and stubbed or local Sheets and Slides. Naming is layered on purpose so HUD chrome, fiction, and shipped software do not blur into one marketing word.

The core loop is deliberately slow and single-threaded. One Node process owns wall time. At a tick interval drawn from roughly 90–120 seconds (configured mid around 105 s), the orchestrator holds a lottery among four employees, assembles a small context, makes at most one OpenRouter call when live, executes a handful of tools, and writes events to an append-only log. The display is one web page: a quiet top bar, a locked-frame 2D office on the left, and an iframe of the last green `dist/` on the right. Chromium in kiosk mode on a Raspberry Pi paints that page to HDMI. Raspberry Pi Connect provides remote watch without opening a forwarded port.

Four load-bearing rules, carried forward from the studio design spec and locked in the Stage 1 repository, organize everything else:

1. **The event log is the truth.** A sprite walks because an event happened. The office is `data/events.jsonl` plus `office.json`, not ambient motion.
2. **Never show a broken build.** The product pane never points at the working copy. Only a promoted green dist/ is framed.
3. **Bots edit data, never machinery.** Employees may write product HTML, office layout, backlog, strategy, journals, and relationships. They cannot write `src/`, `public/`, `config`, or `secrets`.
4. **Secrets stay in the environment.** The OpenRouter key never enters the workspace employees can `read_file`.

A fifth discipline, aimed at the human operator rather than the code, is not to rescue the cast with silent fixes or endless credit. Stage 1 encodes the money side as a hard five-dollar daily ceiling rather than a full credits economy; the ceiling still makes spend a visible constraint on the HUD.

Why write this down as a paper. Fishbowl is small enough to run on a Pi 3-class memory envelope, but it sits at the intersection of several research and systems questions that are usually treated separately: multi-agent orchestration under a hard budget; agent tooling that is structured rather than screenshot-driven; physical kiosk deployment; and an honesty condition between what viewers see and what the system did. The sister paper, *The Interface Is a Variable*, measures how interface representation changes cost and reliability when an agent operates a fixed store. Fishbowl does not rerun that experiment. It applies the structured-tools side of that argument to a living office: employees act through schemas and an event log, while humans watch a canvas and a gated product pane.

Contributions.

1. A concrete Stage 1 architecture for an event-log-truthful multi-agent office: tick lottery, narrow tools, green-build gate, split-screen kiosk, dry-run/replay, and a five-dollar UTC-day ceiling, as implemented in the public Stage 1 repository.
2. An argument, grounded in Pi memory, power, HDMI watchability, and Connect remote access, for why a Raspberry Pi is the right Stage 1 host rather than a laptop lid or a cloud VM alone.
3. A secrets threat model that keeps API keys outside bot-readable paths, with `systemd EnvironmentFile` loading and loopback bind as default exposure posture.
4. A measurement plan over observables the system already emits (ticks, events, spend, green builds, rejects) without inventing success rates or token tables that have not been swept.

Research questions.

1. **RQ1.** Does binding the display strictly to an append-only event log change what a cold viewer can correctly report after five minutes, relative to ambient “busy office” animations?
2. **RQ2.** Under a hard five-dollar daily ceiling and ~90–120 s ticks, which combinations of model tiering, context trimming, and dry-run/replay keep the cast useful without exhausting the budget before a human would choose to leave the display on?
3. **RQ3.** Does the green-build gate (syntax check, candidate iframe, promote-or-reject) keep the right pane honest enough that product-pane drift is rare relative to office-pane event drift?

4. **RQ4.** Relative to driving work through screenshots of the office or product UI, do structured tools plus an event log reduce illegal or machine-touching actions in this deployment setting?

Matching hypotheses, stated as design expectations rather than proven results: H1, event-tied motion is necessary for the five-minute watch test to pass. H2, the five-dollar ceiling plus tiered models leaves headroom at the configured tick mid if live spend tracks the budget model. H3, broken working copies never appear in the product iframe when the gate is intact. H4, structured tools dominate screenshot actuation for employee work on this stack. H1–H4 are not claimed as measured outcomes in this manuscript.

How the paper is organized. Section 2 fixes terms. Section 3 positions the work. Section 4 describes the system. Section 5 explains the Pi choice. Section 6 covers architecture and secrets. Section 7 states what we measure and what we do not yet have numbers for. Section 8 discusses implications and the link to structured agent surfaces. Section 9 is how to ship and run. Sections 10 and 11 cover limitations and future work. An appendix sketches reproduction. References are verified primary sources only.

2. Terms used in this paper

We keep names consistent with the Stage 1 repository and the product naming lock.

- **Fishbowl.** The project and Stage 1 deployment name (repo chrome, systemd comments, ops notes). Not the in-world studio brand.
- **Meridian Desk.** The fictional studio name used in cast speech and office fiction.
- **Meridian Office.** The flagship product shown in the right pane (Docs first).
- **Tick.** One world-clock cycle: select one employee, optionally one model call, tools, events.
- **Event log.** Append-only data/events.jsonl; durable truth for motion and history.
- **Green build / dist/.** Last product tree that passed validation; only this is framed on the right.
- **Working copy.** workspace/product/ and related editable data; never shown as the live product iframe source.
- **Dry-run.** Default mode: scripted turns, real events and walks, zero API spend.
- **Replay.** Walk an existing event log with no model calls.
- **Live.** DRY_RUN=false with an API key; billed turns; pause at the daily ceiling.
- **Daily ceiling.** Hard stop at five USD of recorded spend per UTC day; HUD shows the studio asleep; room dims; log may still replay.
- **Cast.** Nova Chen (programmer), Kessler Holt (QA), Mira Sol (producer), Jules Park (office manager).
- **Turn lottery.** Weighted selection of who acts on a tick (Stage 1 weights favor the programmer seat).
- **Kiosk.** Chromium full-screen on HDMI, pointed at loopback HTTP, managed by systemd.
- **Connect.** Raspberry Pi Connect screen share and remote shell for human watch and break-glass.
- **Sister paper.** *The Interface Is a Variable*—controlled comparison of screenshot, accessibility tree, flat tools, and view-document representations for agents on a fixed application.

3. Related positioning

Most multi-agent demos optimize for chat transcripts or dashboard metrics. Fishbowl optimizes for a watchable room whose motion is accountable to a log. Most computer-use agents optimize for operating a human GUI through pixels or trees. Fishbowl’s employees do not operate the office by clicking the canvas; they call tools, and the canvas follows the log. That is a deliberate inversion of the screenshot-agent stack, and it is the light bridge to the sister paper’s finding that human page channels are expensive and brittle when the alternative is a structured surface.

We do not claim novelty for every ingredient. Event sourcing, kiosk Chromium, systemd supervision, and tool-calling agents are each well trodden. The claim is the Stage 1 composition under explicit honesty rules, a hard spend ceiling, a green-build product pane, and a Pi-first physical watch path, described carefully enough to reproduce and to instrument.

Table R1. Positioning against neighboring strands (verified sources only).

Strand	Representative work	What it does	How Fishbowl differs
GUI / computer-use agents	Claude computer use; UI-TARS; SeeClick	Perceive pixels or GUI structure; emit clicks	Employees act via structured tools; humans watch a derived canvas
Web agent benchmarks	WebArena; Mind2Web; VisualWebArena	Vary agents on fixed human sites	We fix a private office stack and ask whether the display stays truthful
Agent tool protocols	Model Context Protocol (MCP)	Standardize tool catalogs and hosts	Fishbowl’s tool surface is app-local and paired with an event log + green gate
Agent UI measurement	Sister paper (view document vs tools vs human channels)	Treat interface representation as the variable	Fishbowl deploys the structured-tools side in a living kiosk; no sister-store task reruns here
Pi remote access / kiosk	Raspberry Pi OS docs; Raspberry Pi Connect	SSH, screen share, always-on appliance patterns	Applied as the primary Stage 1 watch surface for a multi-agent office

Agent-to-human UI protocols such as A2UI point in the opposite direction from Fishbowl’s employee tools: they let an agent generate interface for a person. Fishbowl’s right pane is human-facing product UI generated by the studio’s own writes, gated for greenness. MCP remains the closest protocol analog for the employee tool catalog, but Fishbowl is not an MCP host in Stage 1; tools are local to the orchestrator. The sister paper’s view-document condition is a measurement probe for application-to-agent state; Fishbowl’s analogous “document” for humans is the event-projected office plus the gated dist/, not a JSON affordance list shown to Nova.

What this paper does not claim: that multi-agent offices are new; that OpenRouter is required forever; that Stage 1 already includes hiring, a full credits economy, or a public stream; or that any success-rate table in this PDF was measured on a controlled sweep. Those absences are intentional.

4. System design: what Fishbowl is doing

4.1 World clock and one employee per turn

A single Node process is the world clock. The tick interval is drawn between configured bounds (default 90–120 s; mid 105 s in the Stage 1 config). One employee acting at a time is deliberate. It keeps the office readable on a small screen, makes file conflicts structurally impossible, and keeps token spend linear in wall-clock time rather than in headcount. Parallel agent storms are entertaining in logs and illegible on HDMI.

On each tick the orchestrator: (1) checks pause state and the daily ceiling; (2) selects an employee by lottery; (3) assembles a small context; (4) in live mode, makes at most one model call; (5) executes returned tool calls under schema and path allowlists; (6) appends events; (7) pushes updates over a websocket to connected displays.

4.2 Context assembly and tools

Per-turn context is intentionally narrow: role and persona, constitution, strategy, backlog, last ten events, relationship opinions, and at most one file under edit. The constitution and stable role text are candidates for provider prompt caching when the endpoint honors cache control. Whole-file rewrites are preferred under a few hundred lines; the product line-count ceiling exists to force ship-and-restart rather than endless patch loops.

Stage 1 tools include `read_file`, `write_file`, `add_task`, `close_task`, `say`, `journal`, `request`, `edit_office` (Jules only), and `edit_self_aesthetics` (self only). Employees never receive tools that edit orchestrator source, renderer machinery, or secret files.

4.3 Cast, models, and conflict without cartoon prompts

The locked cast is four people on distinct model families so disagreement is not one model in four hats:

Table C1. Stage 1 cast (from repository README; model ids are config, not hardcoded callsites).

ID	Name	Role	Accent	Model family seat
nova	Nova Chen	Programmer	Coral	Qwen coding seat (qwen/qwen3-coder-next)
kessler	Kessler Holt	QA	Teal	Nous / Hermes mid seat
mira	Mira Sol	Producer	Amber	Z.ai flash producer seat
jules	Jules Park	Office manager	Indigo	Meta Scout characterful seat

Priorities conflict by construction (ship tonight versus reject loose greens; thrift furniture versus vanity desks). A relationship matrix stub on disk moves on rejects and green ships and decays toward zero across UTC days so grudges do not freeze the show. Prompts describe reasonable professionals; they do not instruct anyone to “be competitive.”

Lottery weights in Stage 1 favor Nova for building, with Mira and Kessler sharing planning and QA mass, and Jules keeping a smaller office-manager weight. Exact weights live in `studio.config.json` and may change; the paper treats them as configuration, not as experimental factors already swept.

4.4 Event log as truth

Every consequential act becomes an event line. The left pane paints from `office.json` plus the event stream. Walk lerps target real objects. Idle means seated at a desk, not a floating “thinking” pose. `say()` becomes a short tailed bubble after markup stripping and a two-line cap. If the websocket is down, a local seed still paints so HDMI is never a void; reconnect uses exponential backoff.

The log is written to survive dirty shutdown. Corrupt or half-written JSON lines are skipped and copied aside so systemd restart after a power cut does not crash the world clock. That detail matters on a Pi more than on a laptop that is rarely yanked from power.

4.4.1 Office dollhouse and presentation rules

`office.json` seeds desks, a break room, meeting table, coffee, couch, whiteboard, windows with floor light, shadows, and light clutter. Objects advertise actions (plan, hang out, break, review) even when social turns are rare, so the room reads as a workplace rather than a waiting room. Jules alone may `edit_office`; aesthetic self-edits are self-only and schema-validated against a wardrobe vocabulary. Clicking a person or the board inspects state. The camera never pans to follow a speaker—the room is the unit of attention.

Dry-run’s scripted evening is itself a readability fixture: a short kanban/review arc that exercises walk-to-desk, pair review at the table, board rewrites, and coffee adjacency without spending tokens. That fixture exists so HDMI bring-up does not depend on a live key.

4.5 Split screen and the green-build gate

The page is one document with a quiet top bar (day, headcount, burn versus five dollars, who is acting, pause control), a left office (~56%), and a right product iframe (~44%). The right pane loads only `/dist/`. After a write under the product workspace, the orchestrator syntax-checks HTML/JS without evaling employee code on the host. If a display client is connected, a hidden iframe loads a candidate URL and reports pass or fail over the websocket. On pass, files copy to `dist/` and a `build_passed` event fires. On fail, `dist/` does not move; QA receives a failure event the ticker can render in English (“Kessler rejected Nova’s write”).

This reuses the Chromium already paid for in RAM. A second headless browser for validation would be a poor fit for a ~905 MiB Pi 3-class host.

The gate’s separation of concerns matters for RQ3. Syntax failure never promotes. Runtime failure in the candidate iframe never promotes. QA narrative events make rejects part of the show instead of silent log lines. The working tree may be temporarily ugly; the audience pane must not be. That asymmetry is how “never show a broken build” stays compatible with unsupervised writes.

4.6 Flagship product: Meridian Office

Stage 1’s flagship is Meridian Office rather than a one-shot utility. Docs is a usable cream-paper editor (structure, preview, local persistence, download). Sheets and Slides exist as local or stubbed surfaces depending on the build cut; the catalogue still holds smaller tools such as Timezone Buddy, units, and kanban for portfolio breadth. The HUD line prefers “Building: Meridian Office — Docs” so studio fiction and product chrome stay distinct. Permanent bans for products the studio ships unattended include

accounts, payments, uploads, and embedded chat—not because models cannot sketch them, but because unattended responsibility for other people’s data at 03:00 is out of scope.

4.7 Dry-run, replay, live, and the five-dollar ceiling

Default boot is dry-run: scripted turns produce real events and motion at zero API cost. Replay mode walks events.jsonl for visitors. Live mode requires an explicit flag and a key; estimated USD per turn accumulates in data/spend.json. At five dollars in a UTC day, ticks stop, the burn bar fills, lights dim, and the HUD reports the studio sleeping. Local stubs may still template a say() so the room does not die silently. The next UTC day resets the counter. This protects the payment instrument; it is independent of any later Stage 2 credits fiction.

A kill switch pauses without deleting the log: HUD pause, POST /api/pause, a data/PAUSED marker, or a local /kill bookmark. While paused, the office can slowly replay recent events so Connect still looks like a room.

4.8 Five-minute watch acceptance

Product acceptance for Stage 1 is a human watch test, not a leaderboard. On Connect or HDMI, cold start, sound off, after five minutes a stranger should name the people, name Meridian Office, say whether spend is under five dollars, and say whether they would leave the feed open. Failure of office feel is product failure even if the orchestrator is correct: floating thinkers, broken-build iframes, JSON tickers, or a metrics dashboard aesthetic all fail the bar.

5. Why a Raspberry Pi is needed

Stage 1 could run on a laptop or a cloud VM. The Pi is not required for Node to parse JSON. It is required for the Stage 1 thesis: a physical, always-on, HDMI-watchable appliance that makes the honesty rules hard to dodge.

5.1 Always-on kiosk and physical watchability

A laptop lid closes. A cloud VM has no inherent spectator surface; someone must open a browser on purpose. A Pi on a desk, wired to a display, with Chromium in kiosk mode pointed at http://127.0.0.1:PORT, is a television channel for the studio. Useful utilization is “always show what the studio is doing,” including dry-run evenings when no key is loaded. Raspberry Pi Connect then adds remote eyeballs without redesigning the UI.

5.2 Cost and power envelope

The hardware is already owned in the pilot setting. Power draw is appliance-class rather than workstation-class. That matches a 24/7 process that spends most of its wall time waiting on a 90–120 s timer. The expensive part of Fishbowl is API tokens, not CPU. Keeping the host cheap focuses attention on the ceiling that actually matters.

5.3 Constrained RAM as design pressure

A Pi 3-class device reports on the order of ~905 MiB available RAM in the pilot notes. Chromium plus Node on that budget is tight but instructive. The Stage 1 renderer forces device pixel ratio 1, paints the room into an opaque canvas and blits, stops requestAnimationFrame while everyone is seated, avoids

WebGL and draw workers, and ships kiosk flags such as `--disable-gpu`, `--disable-dev-shm-usage`, and a limited renderer process count. Products stay 2D/HTML with local system fonts. These are not aesthetic preferences first; they are memory preferences that happen to produce a readable office.

If the compositor still swaps, a lite query flag drops leftover motion. Soundtrack stays off by default—sample files and decode cost are unwelcome. zram is preferred over heavy disk swap. The checkout prefers a USB SSD when possible because the event log is many small writes, which SD flash dislikes over months.

5.4 Loopback bind, systemd, and Connect

The HTTP and websocket server default to loopback. The kiosk on the same machine is the primary client. Remote humans use Connect (or later Tailscale / a tunnel)—not a casually forwarded port. Two systemd units cover the orchestrator (`Restart=always`, optional secrets `EnvironmentFile`) and the kiosk (after graphical target, Wayland/ozone flags on Bookworm labwc stacks). Screen blanking and DPMS are disabled so overnight watch does not become a black rectangle.

5.5 Why not laptop or cloud VM alone for Stage 1

A laptop is a poor 24/7 kiosk: sleep, lid, browser updates, and personal-session contention fight the appliance story. A cloud VM is an excellent later home for the orchestrator when uptime and SD wear matter for a public audience, and the design spec already anticipates splitting brain and display. For Stage 1, starting on the Pi forces the honesty surface to exist: if it is not interesting on HDMI, it is not done. Moving the orchestrator later is a clone and a key; inventing a watchable room later is harder.

Honest caveats belong here too. Home power cuts, low-voltage power supplies, and SD corruption are real. The pilot documents low-voltage remediation before true unattended overnight. Stage 3-style “Pi as window only” remains available if OOM wins.

5.6 Operator environment surface

The Stage 1 process reads a small env surface. Defaults keep the appliance safe to boot cold: `dry-run` true, loopback host, port 8787, daily ceiling five USD, local stubs true when sleeping, tick bounds 90–120 s. Live spending requires both a key and an explicit `dry-run` false. Replay is a mode flag that ignores the model path entirely. A lite flag drops motion for memory recovery. Optional OpenRouter referer/title headers identify the app to the provider without placing secrets in the workspace.

Table E1. Selected environment variables (defaults from repository README).

Variable	Default	Role
DRY_RUN	true	Live only when false and a key exists
STUDIO_MODE	empty	replay walks the log at \$0
OPENROUTER_API_KEY	empty	Required for billed turns
DAILY_CEILING_USD	5	Hard pause threshold
HOST / PORT	127.0.0.1 / 8787	Loopback studio bind
STUDIO_TICK_MIN_MS / MAX	90000 / 120000	Lottery interval bounds
STUDIO_LITE	empty	Memory-saving motion cut

6. Architecture and secrets threat model

6.1 Process and data layout

Machinery lives under `src/` and `public/` and is not writable by bots. Editable data lives under `workspace/` (office layout, backlog, personas, product sources). `dist/` holds the last green build. `data/` holds the event log, spend file, and pause marker. `deploy/` holds `systemd` units. `studio.config.json` locks cast, models, tick bounds, and ceiling. Snapshots are a hook for later weekly digests, not a Stage 1 audience site.

Table A1. Trust boundaries (Stage 1).

Zone	Writable by employees?	Notes
<code>workspace/ data</code>	Yes, schema-validated	Product HTML, <code>office.json</code> , journals, relationships
<code>src/, public/, config</code>	No	Orchestrator and renderer machinery
<code>dist/</code>	Only via green gate	Promotion is orchestrator-side
<code>~/.secrets/fishbowl/</code>	No	Outside workspace; mode 0600
Host Node process	N/A	Never executes employee JS via <code>eval</code>

6.2 Secrets

The OpenRouter API key lives under a home secrets directory (for example `~/.secrets/fishbowl/openrouter.env`), mode 0600, loaded by `systemd` with an optional `EnvironmentFile=-...` so dry-run boots without a key. Application code reads `process.env` only, fails closed if live mode lacks a key, and never logs the value. The key must not sit in git, chat, durable agent memory, or any path covered by employee `read_file`.

Rotation is operationally boring on purpose: create a new key, overwrite the env file, restart the unit, revoke the old key. The browser remains the sandbox for product code: the worst candidate-iframe outcome is a broken page, which the gate already turns into a QA event.

6.3 Output filtering and pause

`say()` is stripped and capped before it becomes a bubble. The kill switch pauses the world from HUD, API, file marker, or phone bookmark on loopback. These are Stage 1 output filters for a private pilot—not a claim that the system is safe to point at the open internet with unattended social posting.

6.4 Structured tools versus screenshot driving

Employees do not ground “click the whiteboard” in pixels. They call tools; the renderer projects consequences. That choice aligns with the sister paper’s structured conditions: flat tools are cheap per call once schemas are paid for, and they avoid the grounding tax of screenshots. Fishbowl adds two application-specific honesty mechanisms the sister store study did not need: an event log that drives a human-facing canvas, and a green-build iframe gate so spectators never see a broken working tree. We do not import the sister paper’s task-success or token tables into this manuscript; the link is architectural, not numerical.

7. What we measure

Fishbowl already emits artifacts that support measurement without a separate fantasy harness: tick timing, event lines, spend estimates, pause and sleep states, build_passed / build_failed events, and websocket connectedness. The correct next step is to log and summarize those observables across dry-run and live windows.

Table M1. Observables and status (no fabricated sweep numbers).

Observable	Source	Status in this paper
Tick interval realized	Orchestrator timing	Design target 90–120 s; no multi-day histogram published here
Events per hour / by type	<code>events.jsonl</code>	Collectable; not tabulated as an experiment
UTC-day spend versus \$5	<code>spend.json</code>	Mechanism implemented; live week-long burn not claimed
Green vs failed builds	Gate events	Mechanism implemented; rates not swept
Five-minute watch pass/fail	Human protocol	Acceptance criterion defined; formal rater study not run
Illegal path writes blocked	Tool allowlist logs	Planned metric; no fake precision

Budget model versus measurement. Internal planning notes project roughly one dollar per day of live spend at locked model ids, ~105 s ticks, and stated lottery weights, with headroom to the five-dollar ceiling. That projection is a planning arithmetic from catalog prices and assumed token sizes. It is not a substitute for a recorded live week on the Pi, and this paper does not promote it to an experimental result. Dry-run and replay remain the default until a human explicitly enables live mode.

What we refuse to invent. Task success percentages, token tables by employee, bootstrap confidence intervals, and “N-day uptime” claims do not appear as data here. If a later revision adds them, they will come from dated log artifacts and a written sweep protocol, in the same spirit as the sister paper’s insistence on execution grading and finished traces.

Planned measurement protocol (not yet executed as a published sweep). A reproducible window would (i) run 72 hours of dry-run on the Pi with kiosk attached, (ii) export `events.jsonl` and `systemd` journal slices, (iii) enable live for a bounded UTC day under the five-dollar ceiling, (iv) compute tick histograms, event-type mixes, spend curves, and green/fail counts, and (v) run the five-minute watch test with at least two raters who were not the implementers. Until those artifacts exist with dates, tables of rates do not belong in the results section of this paper.

Instrumentation already present. The status line and `/api/state` surface day number, burn, who is acting, and sleep/pause. Events name actors and tool outcomes in English suitable for a ticker. Spend is estimated per turn rather than trusted from a provider dashboard alone, so the ceiling can trip even if an external console is unread. These are the raw materials for Table M1; they are not themselves a completed experiment.

7.1 Design budget arithmetic (planning only)

Planning notes produced for Stage 1 model selection use catalog prices as of a 2026-09-20 snapshot, a mid tick of 105 s (~823 ticks per 24 h), and lottery mass concentrated on the programmer seat. Under those assumptions the notes project on the order of one dollar per live day with headroom to five dollars. We restate that arithmetic only to explain why the ceiling and tiering were considered jointly feasible. Prices

move; token sizes move; Jules's weight and max tokens move. None of that arithmetic is a measured burn from the Pi's `spend.json`.

8. Discussion

Fishbowl's central bet is that truthfulness is a systems property, not a prompt flourish. If motion can happen without events, viewers learn to discount the room. If the product pane can show a working tree, viewers learn to discount the ship. If employees can read the API key, the cast becomes an accidental exfiltration agent. The four load-bearing rules are therefore not slogans; they are the minimal set that keeps Stage 1 from rotting into theater.

The Pi bet is related. Constraint forces the UI to stay cheap and the deployment to stay watchable. The same constraint also risks OOM and SD wear; documenting memory flags and SSD preference is part of the research surface, not an embarrassment to hide.

Relative to computer-use agents that operate human software through screenshots, Fishbowl asks employees to live inside a purpose-narrowed tool world and asks humans to watch a projection. That is closer to building an application-for-agents than to scraping a GUI-for-humans. The sister paper measures representation on a fixed store; Fishbowl shows what a structured, gated, log-backed application looks like when the "application" is a small software studio that ships into its own right pane.

Stage 1 also declines several tempting expansions: public stream encoding on the same machine, hiring interviews, full credits enforcement, isometric art, and WebGL. Hooks exist (snapshots/, relationship file, office layout). Living with the fishbowl first is the stated build order: learn whether the room holds attention and what the real bill looks like before building the company around it.

8.1 Failure modes the design anticipates

The studio design spec lists behavioural failure modes that Stage 1 only partially covers, and that is intentional. Blandness—the tendency of models to converge on polite managerial prose—is fought with distinct model families and conflicting priorities rather than with a prompt that says "be spicy." Drift toward meta-work is fought with a product that must remain pokeable and a green pane that makes non-shipping obvious. Degenerate conversational loops are a Stage 2+ detection problem (hash the last events; force a different action). Grudge saturation is already addressed by relationship decay. Soap-opera capture is constrained by lottery weights that still favor building. Vocabulary starvation (requests for undrawable props) becomes an art backlog when rejects are logged.

Systems failure modes are more immediate on a Pi: OOM from Chromium, SD wear from tiny writes, low-voltage throttling, Wayland session loss after reboot, and websocket flaps. The seed-first paint, corrupt-line quarantine, linger for Connect, and SSD preference are mitigations aimed at those, not at abstract multi-agent theory.

8.2 Remote access without confusing the threat model

Human operators have three different jobs that must not collapse into one channel. Connect screen share answers "what does HDMI look like right now?" Connect remote shell answers "I need a break-glass terminal without LAN." SSH on the LAN answers "unattended git, apt, journalctl, and rsync." Automating Connect screen share with another agent loop would burn tokens and fight the kiosk; the ops notes explicitly discourage that. File shares (Samba/NFS) and VNC are optional and unnecessary for Week 1.

Nothing in the product pane should require a public ingress; if WAN viewing beyond Connect is needed later, a tunnel or mesh VPN is the documented posture.

8.3 What “office feel” contributes that dashboards do not

A metrics strip can show spend and headcount more densely than a dollhouse. Fishbowl still chooses desks, walk lers, speech bubbles, and a fixed camera because the acceptance test is whether a stranger leaves the feed open. The workplace-aquarium metaphor is load-bearing: warm planks, window light, soft shadows, accent rings, and English event lines are not decoration on top of an orchestrator—they are the interface through which honesty is judged. If the cast floats while “thinking,” the event-log rule has already failed visually even if the JSONL is perfect.

9. How to ship and run

The Stage 1 repository documents the operator path; this section summarizes it at paper density.

- **Host.** Raspberry Pi OS Debian aarch64; heatsink/fan; prefer USB SSD for the checkout.
- **Node.** User-local Node 20 LTS arm64 tarball (pilot pin v20.20.2) on PATH; no requirement for `/usr/local` sudo symlinks.
- **Install.** Clone, `npm install`, `npm start`. Default dry-run needs no key.
- **Display.** Open loopback HTTP (default port 8787) or enable the Chromium kiosk unit with Wayland/ozone settings appropriate to Bookworm labwc.
- **Secrets.** Only when going live: write `~/.secrets/fishbowl/openrouter.env` mode 0600; set `DRY_RUN=false`; restart the orchestrator unit.
- **Remote watch.** Raspberry Pi Connect for screen and break-glass shell; LAN SSH key-only for unattended maintenance; do not forward the studio port casually.
- **Deploy loop.** `git pull`, reinstall units if changed, `daemon-reload`, restart orchestrator and kiosk, read `journalctl`.

Fast local loops may shorten tick bounds for development; the Pi default stays slow enough to watch and cheap enough to leave on.

10. Limitations

Stage 1 is a private pilot. It is not a public stream platform, not a hiring marketplace, and not a full studio economy. The cast size is four; the tool surface is narrow; the product scope is intentionally small. The green-build gate depends on a connected display client for the candidate iframe path; headless promotion policy beyond syntax check must stay explicit in code review.

Memory headroom on Pi 3-class hardware is a standing risk. Live spend has a hard ceiling but still requires human judgment before enabling billing. Model ids and prices move; any budget arithmetic ages. Dry-run scripted turns demonstrate motion and gating but do not prove that live models will keep the five-minute watch test green.

This manuscript itself contains no controlled experimental tables. Until dated sweeps exist, claims about reliability remain architectural. The sister paper’s numerical frontier is a different artifact and must not be silently reused as Fishbowl evidence.

11. Future work

Nearest work is measurement: multi-day tick and spend histograms on the Pi; green-fail rates by employee; illegal-path attempt counts; a small formalization of the five-minute watch test with more than one rater. Next systems steps already foreshadowed in the design spec include a credits economy, hiring, richer relationships, and moving the orchestrator to a small VPS while leaving the Pi as the HDMI window.

A public stream / video platform path is planned and not built: a second machine should subscribe to the event stream and encode, rather than asking the world clock host to drop ticks because someone is watching. Weekly snapshots for a calendar UI are hooks, not Stage 1 deliverables. GPIO pause buttons and richer wardrobe economies can wait until the room has been lived with.

On the research bridge, a later study could compare—within Fishbowl—employee success when product inspection is offered as structured status versus as screenshots of the right pane, without disturbing the event-log truth rule for the office canvas. That experiment should be designed with the sister paper’s matched-grain discipline, not as a casual demo toggle.

Appendix: reproducing the Stage 1 fishbowl

Public Stage 1 code: <https://github.com/akashnaren/raspberry-pi-fun>. Clone on an aarch64 Pi or any Node 20+ host for dry-run. Confirm `DRY_RUN=true` by default, open the loopback page, and confirm: (1) office seed paints before websocket connect; (2) events append under `data/`; (3) right pane serves `dist/`; (4) pause dims and does not erase the log; (5) with a key and live flag, spend accumulates and sleeping triggers at five USD per UTC day. Kiosk units live under `deploy/`. This manuscript rebuilds from HTML to PDF via Chromium headless print.

Flow diagram companion (operator one-pager): event lottery → context/model → tools → event log → left office / right green product → daily ceiling → future stream placeholder.

Appendix B: green-build sequence (operator checklist)

1. Employee writes under `workspace/product/` via tool.
2. Orchestrator syntax-checks without host eval.
3. Connected display loads candidate in a hidden iframe; reports pass/fail on the websocket.
4. Pass copies to `dist/` and emits `build_passed`; fail leaves `dist/` untouched and emits failure for QA.
5. Right-hand iframe continues to serve only `/dist/`.

Appendix C: naming layers (do not collapse). Fishbowl is the project. Meridian Desk is the studio fiction. Meridian Office is the flagship product. Timezone Buddy remains catalogue utility naming. HUD chrome stays quiet—no shouty project wordmark across the office.

References

Verified sources only. Each entry was checked against primary documentation or a stable paper page. No other citations are asserted.

Akash Premkumar. The Interface Is a Variable: Measuring the Cost and Reliability of Purpose-Built UI Representations for LLM Agents. Research note / sister paper hub:
<https://akashnaren.github.io/research/agent-native-ui/> ; code: <https://github.com/akashnaren/agent-ui-metrics>.

Fishbowl Stage 1 repository (raspberry-pi-fun). <https://github.com/akashnaren/raspberry-pi-fun>.

Raspberry Pi Documentation — Remote access.

<https://www.raspberrypi.com/documentation/computers/remote-access.html>.

Raspberry Pi Documentation — Connect.

<https://www.raspberrypi.com/documentation/services/connect.html>.

Model Context Protocol. <https://modelcontextprotocol.io/>.

OpenRouter documentation. <https://openrouter.ai/docs>.

WebArena: A Realistic Web Environment for Building Autonomous Agents.

<https://arxiv.org/abs/2307.13854>.

Mind2Web: Towards a Generalist Agent for the Web. <https://arxiv.org/abs/2306.06070>.

VisualWebArena: Evaluating Multimodal Agents on Realistic Visually Grounded Web Tasks.

<https://aclanthology.org/2024.acl-long.50/>.

Anthropic: Introducing computer use. <https://www.anthropic.com/news/3-5-models-and-computer-use>.

UI-TARS: Pioneering Automated GUI Interaction with Native Agents. <https://arxiv.org/abs/2501.12326>.

SeeClick: Harnessing GUI Grounding for Advanced Visual GUI Agents. <https://aclanthology.org/2024.acl-long.505/>.

Node.js distributions (arm64 tarballs). <https://nodejs.org/dist/>.